

Asclepius: Medication and Wellbeing Tracker

Skills Roadmap and Implementation Architecture

Version: 1.0

Prepared for: Thabiso Camilo Christopher

Date: June 12, 2026

Status: Planning document. No code exists yet, and per your standing rule, none will be written for you. This document gives you the skills map, the architecture, the phase plan, and every compliance, distribution, and portfolio consideration required to make this real. You build it.

Codename note: Asclepius is a placeholder, not a decision. The rod of Asclepius (a single serpent on a staff) is the correct symbol of medicine; the caduceus (two serpents, winged) belongs to Hermes and was adopted by mistake. Replace the codename whenever you choose a real name; Part 8.5 covers the naming process.

Part 1: Purpose and Strategic Fit

1.1 What this is

A medication and wellbeing tracking system built in Python, progressing from a command-line tool to a full web application, with a native iOS companion as the final phase. The system tracks user-defined medications and schedules, guards against double dosing, alerts when a dose window passes without a log entry, records physical, mental, and emotional state on configurable 1 to 5 or 1 to 10 scales, produces statistics and graphs, ingests Apple Health data, and generates printable practitioner reports over any custom date range.

1.2 Who it serves

Three roles, and keeping them distinct will shape every design decision:

- **The patient (Lerato).** The daily user. She logs doses and wellbeing entries, receives reminders, and needs the friction of logging to approach zero. She is also a medical doctor, which makes her an unusually qualified first user: she can tell you exactly what a practitioner report needs to contain and what makes one useless.
- **The practitioner.** Never touches the app. Consumes a printed or PDF report. The report is a product in its own right and gets its own specification (Part 6).
- **The builder (you).** This is your Python proving ground. Every subsystem maps onto a skill the Sovereign Creator Roadmap already demands.

1.3 Fit with the Sovereign Creator Roadmap: no new front

The roadmap's Risk 1 is divergence, and the One Big Thing rule says new fronts open only at quarter boundaries by closing another. This project passes that test because it is not a new front. It is the existing front given a real target:

- **It occupies the Python study hours, not new hours.** Q3 2026's One Big Thing is the dual spine: RPIE Python plus Python Crash Course, plus the Google Cybersecurity Certificate. PCC's prescribed path is chapters 1 to 11 plus two Part II projects. This

tracker absorbs the role of the Data Visualization project (PCC chapters 15 to 17 teach exactly the matplotlib and Plotly skills the stats engine needs) and later exercises the same Django muscle the Learning Log project (chapters 18 to 20) teaches. Alien Invasion remains Artifact 5 untouched. You are not adding a project; you are replacing a textbook-shaped project with a real one that someone you love will use.

- **The devlog feeds the audience engine.** The weekly public artifact obligation (2 hours per week) can be served by short build-log posts on this project for many weeks running. A tool built for a physician sister is a genuinely compelling public narrative.
- **The skills are convergence skills.** Parsing Apple Health's export XML is structurally the same skill as Artifact 3's access-log parsing: ingest a messy machine-generated format, normalize it, flag what matters, summarize per entity. Handling sensitive health data with a local-first, encryption-aware, no-third-party posture is a security mindset demonstrated in practice, which you can speak to honestly in content security interviews.
- **The hard boundary.** During Q4 2026, the Security+ closure sprint and the Ring 1 campaign own the calendar. Phase 3 of this project (the web application) is explicitly scheduled to yield to that sprint. The phase plan in Part 5 encodes this.

1.4 Honest portfolio positioning

This is not a Ring 1 content-security artifact and should not be presented as one. What it honestly demonstrates: production Python (classes, persistence, testing, packaging), correctness-critical scheduling logic, data ingestion from a hostile real-world format, data analysis and visualization, report generation, privacy-conscious architecture, and the full arc from CLI to web to (eventually) native mobile. Those are the operational-altitude skills named in Part 5 of the roadmap, evidenced by a shipped tool with a real user. In an interview, the sentence "I built a medication adherence and wellbeing system for my sister, who is a physician, and designed it local-first because health data should not leave the household by default" carries more weight than any certificate line.

1.5 The immediate value move (before any code exists)

Apple's Health app already includes a Medications feature: Lerato can define her medications and schedules and log doses in it today, on the phone she already carries. Apple Health can then export everything she logs (Part 4.6). This means:

1. **She starts capturing data now**, with zero wait on your build.
2. **Your Phase 1 importer gives her value before your UI does.** The first genuinely useful thing you ship can be a parser that turns her Apple Health export into a practitioner report. No data captured during your build period is lost.
3. **You inherit a migration path instead of a cold start.**

This is the recommended opening move. It also de-risks the project: if Apple's built-in feature plus your report generator turns out to satisfy her completely, you have learned that before building a full app, which is the hot dog method applied to your own family.

Part 2: Product Definition

2.1 The seven feature pillars

1. **Medication registry.** User-defined medications: name, strength, unit, form, free-text notes, active or archived status. No drug database lookup in early phases (see 2.3).
2. **Schedule engine.** Per medication, user-defined frequency expressed as either clock-anchored times ("08:00 and 20:00 daily") or interval-anchored rules ("every 8 hours after last dose"), plus as-needed (PRN) medications with no schedule but a minimum-interval guard.
3. **Double-dose guard.** When a dose is logged, the system compares elapsed time since the last confirmed dose of that medication against the user-configured minimum interval. If inside the interval, it warns and requires explicit confirmation, recording the confirmation in the event history. It informs; it never advises (see 2.2).
4. **Missed-dose alerting.** A dose becomes due at its scheduled time, enters a user-configured grace window, then triggers an alert, with user-configured repetition and quiet hours. The delivery mechanism changes per phase (Part 4.5).
5. **Wellbeing tracker.** Three default axes (physical, mental, emotional), each on a per-axis configurable scale of 1 to 5 or 1 to 10, with optional free-text note per entry and support for multiple entries per day. The axis set itself should be user-extensible later (pain, energy, sleep quality) but ships fixed at three.
6. **Statistics and graphs.** Adherence rates per medication and overall; on-time versus late versus missed breakdowns; wellbeing trends per axis; overlay views correlating adherence with wellbeing; all filterable by date range.
7. **Practitioner report.** A printable PDF over any custom date range, specified fully in Part 6.

2.2 The regulatory design constraint that shapes everything

This was verified against current FDA material this session (sources in Part 11). The FDA's Policy for Device Software Functions and Mobile Medical Applications places medication reminder apps under enforcement discretion: software that helps patients adhere to **pre-determined medication dosing schedules by simple prompting**, without providing specific treatment or treatment suggestions, is low-risk territory the FDA does not actively regulate. The design rules that keep this project inside that territory:

- **The user defines every schedule, dose, and interval.** The app never suggests a dose, a frequency, a change, or a drug.
- **The double-dose guard is a schedule-conflict notice, not a safety ruling.** Correct wording: "You logged a dose of X less than your configured minimum interval (8 hours) ago, at 14:05. Log anyway?" Incorrect wording: "Do not take this medication." The first reports the user's own rule back to them; the second is clinical advice.
- **No drug interaction checking, ever, in this project.** Interaction analysis moves software toward clinical decision support, a different regulatory class and a liability you have no reason to carry. If a future version wants it, that is a separate project with separate counsel.
- **Persistent disclaimer.** The app and every report carry wording to the effect of: this software is an organizational tool, is not a medical device, does not provide medical advice, and all data is self-reported; consult a healthcare professional for medical decisions.

2.3 Explicitly out of scope (all phases of this document)

Drug interaction checking. Dose recommendations. Diagnosis or symptom interpretation. Pharmacy or prescription integration. Multi-patient or clinician-facing accounts. Real-time data sharing with third parties. Each of these is a scope, liability, or regulatory escalation. The features queue; the curiosity queues.

2.4 Competitive landscape and why you build anyway

Verified this session: Apple Health includes medication tracking with logging and list sharing, and its data is exportable. Established market apps in this category include Medisafe and MyTherapy; treat any claim about their current feature sets as unverified until you check them directly (flagged in Part 11). Given that competition, the honest reasons to build:

1. **The correlation layer.** Tri-axis wellbeing on configurable scales, correlated against adherence over custom ranges, rendered the way a specific physician wants to read it. General-purpose apps serve averages; you serve one patient and her doctor exactly.
2. **The practitioner report as a first-class product.** Most consumer apps treat export as an afterthought. Yours treats the printed report as a primary deliverable, designed with a physician's direct input.
3. **Data sovereignty.** Local-first, no account, no third-party analytics, household-controlled. For health data, this is a feature competitors structurally cannot match.
4. **The portfolio arc.** Even if Lerato used a commercial app, you would still need a project exactly this shape to learn what Q3 demands. The market does not need to validate a learning vehicle; only a public release does (Part 5, Phase 5 gate).

2.5 Scale configuration detail

Each wellbeing axis carries its own scale definition: minimum (1), maximum (5 or 10), and optional anchor labels for the endpoints ("1 = bedridden", "10 = fully functional"). Scale choice is set per axis at creation and locked once entries exist, because mixing 1 to 5 and 1 to 10 data inside one axis poisons every downstream statistic. If she later wants to change a scale, the correct mechanism is closing the old axis and opening a new one, keeping historical data intact and honest. Record this rule now; it is the kind of decision that is cheap on day one and expensive on day ninety.

Part 3: Skills Roadmap

3.1 What you already hold

From CS50x (December 2025): Python fundamentals, C, SQL, Flask with Jinja templates, HTML and CSS basics, the mental model of request and response. From the June 9 environment setup: a working macOS Python workspace with a project-local .venv, Pylance, Black formatting, Ruff linting, debugpy, and pytest conventions already configured. From the C++ track: disciplined error-handling thinking (pre-conditions, post-conditions, invariants) that transfers directly to the dose-logic subsystem. You are not starting from zero on a single row of the table below.

3.2 The skill tiers

Hour estimates are for focused learning plus first application inside this project, assuming your current level. They are planning estimates, not promises.

Tier	Skill	Where you learn it	What it unlocks here	Est. hours
1	Python core (collections, control flow, functions)	PCC ch. 1 to 8 (in progress)	Everything	In progress
1	Classes and object design	PCC ch. 9	Medication, ScheduleRule, DoseEvent, MetricEntry	6 to 10
1	Files, exceptions, JSON	PCC ch. 10	Config, first persistence, import and export	4 to 6
1	Testing with pytest	PCC ch. 11 (3rd ed.; 2nd ed. teaches unittest, use pytest regardless)	Locking down dose-window logic before trusting it	6 to 10
1	datetime, timezone, zoneinfo	Python docs (not in PCC)	The single most safety-critical subsystem (Part 4.4)	8 to 12
1	sqlite3 module	Python docs plus your CS50x SQL	Real persistence from v0.1	4 to 8
1	argparse CLI design	Python docs	The Phase 1 interface	3 to 5
2	matplotlib	PCC ch. 15	Static charts for reports	6 to 10
2	Plotly	PCC ch. 15 to 17 (3rd ed.)	Interactive charts for the web phase	4 to 8
2	CSV and JSON data handling, real datasets	PCC ch. 16 to 17	Health Auto Export ingestion	3 to 5
2	Jinja2 standalone templating	Held from CS50x Flask; docs	Report HTML templates without a web server	2 to 4
2	WeasyPrint (HTML to PDF)	Project docs	The practitioner PDF	4 to 8
3	XML stream parsing (ElementTree iterparse)	Python docs	Apple Health export.xml at half-gigabyte scale	5 to 8
3	Scheduling (APScheduler; launchd and cron concepts)	Project docs; macOS man pages	Reminder checks that run without you	5 to 8
3	macOS local notifications from scripts (osascript)	macOS docs	Phase 1 desktop alerts	1 to 2
3	Packaging (pyproject.toml, editable installs, pipx)	Python Packaging User Guide	An installable CLI, portfolio polish	4 to 6

3	Type hints plus mypy or Pylance strict	Python docs (environment already set)	Professional codebase signal	ongoing
4	Django (models, auth, admin, forms, templates)	PCC ch. 18 to 20	Phase 3 web app; admin gives a free data-entry UI	25 to 40
4	PWA fundamentals (manifest, service worker)	MDN	Installable web app on her phone	6 to 10
4	Web Push with VAPID	MDN plus a push library	Reminders on iPhone without an App Store app	8 to 14
4	HTTPS deployment basics (a small host, Let's Encrypt)	Host docs	A reachable, secure instance	6 to 10
5	Swift, SwiftUI, HealthKit, UserNotifications	Apple tutorials (2027)	Phase 4 native companion	60 to 100+

3.3 The sequenced path, tied to your actual calendar

1. **Now to PCC chapter 9:** continue PCC as planned. While reading chapter 9, design this project's classes on paper as your worked example instead of the book's. Retrieval practice: close the chapter, write the class responsibilities from memory, then check.
2. **PCC chapters 10 to 11 window:** learn sqlite3 and the datetime/zoneinfo material alongside, because PCC will not give them to you and Phase 1 cannot exist without them. Hard Start rule: the timezone subsystem is the hardest material in Phase 1; schedule it first in each session.
3. **PCC chapters 15 to 17 window:** the stats and charts engine is your Data Visualization project. Same skills, real data.
4. **PCC chapters 18 to 20 window (calendar permitting):** Django via Learning Log, then immediately re-derive the same patterns in this project. Learning Log is a user-plus-entries journal app; this tracker is structurally its sibling, which makes the transfer nearly frictionless.
5. **2027:** Swift and HealthKit, aligned with the roadmap's existing plan that native Apple development enters after the credential keystones land.

3.4 Deep-study topics PCC does not cover (do not skip these)

The timezone and DST material, sqlite3, packaging, APScheduler, WeasyPrint, XML stream parsing, and Web Push are all outside PCC's syllabus. Budget them deliberately (the table above does). Each one is small alone; skipped together they are the difference between a tutorial project and a real one.

3.5 Skills you do not need yet (fronts that must queue)

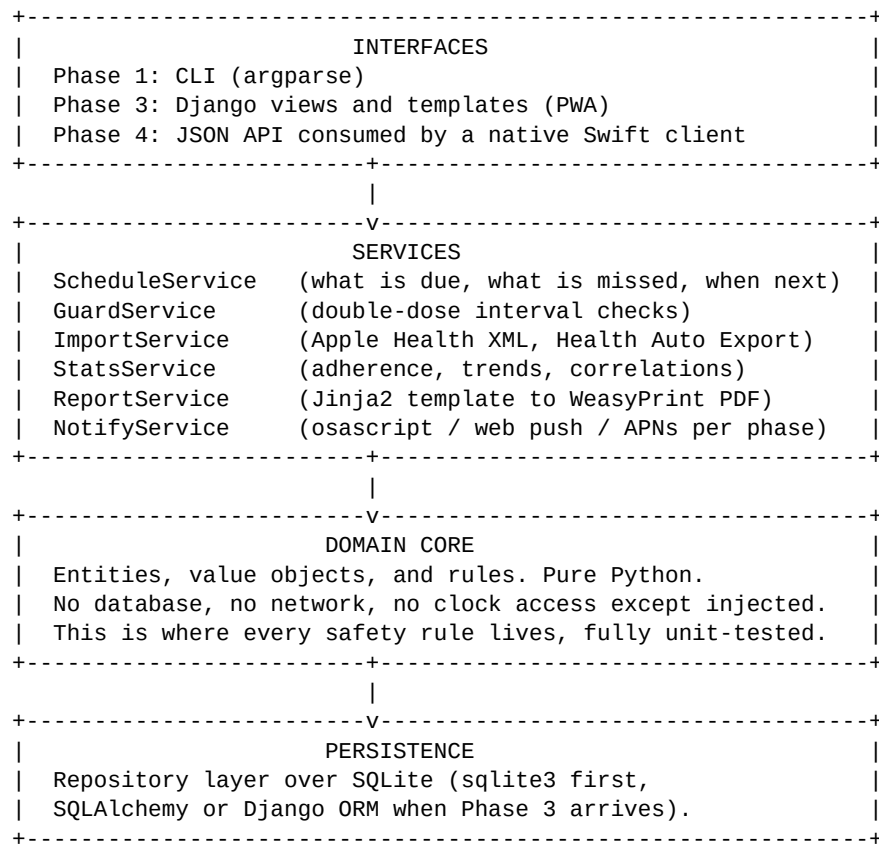
pandas (your aggregations fit in SQL and plain Python at this scale; adopt pandas only if Phase 2 statistics genuinely hurt without it), async Python, Docker, PostgreSQL, React or any JavaScript framework (server-rendered Django templates plus minimal JS is correct for Phase

3), Kubernetes, and machine learning of any kind. Every one of these is a real skill with a real season. None of those seasons is now.

Part 4: Implementation Architecture

4.1 The layered architecture

The same philosophy as your C++ app designs: a pure core that knows nothing about its interface, wrapped by replaceable shells. The interfaces change across phases; the core survives all of them.



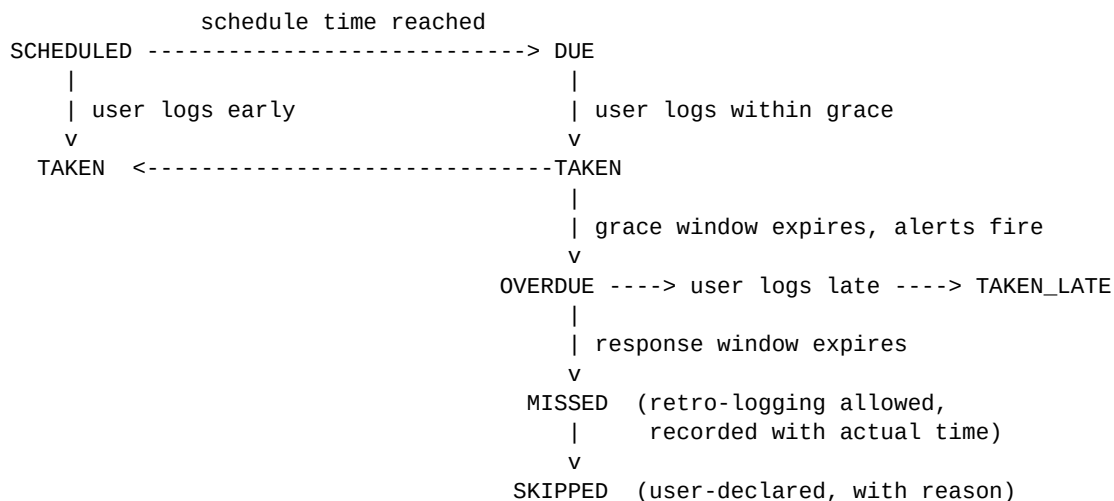
Two rules make this architecture earn its keep. First, the domain core receives the current time as a parameter rather than calling the clock itself; this single decision is what makes the dose logic fully testable ("given last dose at T1 and an attempt at T2, the guard fires"). Second, the repository boundary means swapping sqlite3 for Django's ORM in Phase 3 touches one layer, not the codebase.

4.2 The data model (conceptual; you write the schema)

- **Profile.** One per patient. Display name, home timezone, quiet hours, locale preferences. Even though v0.1 is single-user, modeling Profile from day one keeps Phase 3 multi-user-capable without surgery.
- **Medication.** Name, strength value, strength unit, form, notes, minimum interval between doses (user-set), active flag, created and archived timestamps.

- **ScheduleRule.** Belongs to a Medication. Type: clock-anchored (a set of local wall-clock times plus days of week) or interval-anchored (a duration since previous dose) or PRN (no schedule, guard only). Effective-from and effective-until dates so regimen changes preserve history instead of rewriting it.
- **DoseEvent.** The append-only heart of the system. Medication reference, scheduled-for timestamp (nullable for PRN), actual-logged timestamp in UTC, status (see 4.3), guard-warning-shown flag, guard-overridden flag, optional note, created-at. Never updated except status transitions; never deleted, only marked voided with a reason. Your practitioner report's credibility rests on this table being an honest ledger.
- **MetricAxis.** Name (physical, mental, emotional), scale minimum and maximum, anchor labels, active flag, created-at. Locked scale per 2.5.
- **MetricEntry.** Axis reference, integer value, optional note, logged-at UTC, created-at.
- **ImportedSample.** Source (apple-export-xml, health-auto-export, manual), source record identifier for deduplication, type, value, unit, start and end timestamps, raw payload reference. Imports must be idempotent: re-running an import never duplicates rows.
- **ReportRun.** Date range, filters, generated-at, output path. A small table that turns "can you re-send March's report" into a one-line answer.

4.3 The dose lifecycle as a state machine



Define the states, the legal transitions, and the timestamps each transition records, then test the table exhaustively before building anything on top of it. Note the crossover: this is the state machine pattern from Nystrom's Game Programming Patterns, already on your shelf for the game track. The same pattern that will run enemy AI in Game 1 runs dose lifecycles here. Skills compound when they multiply.

4.4 The four correctness-critical subsystems

1. Time. The hardest and most safety-relevant problem in the entire project, and the reason the datetime study block in Part 3 is not optional. The rules:

- Store every actual event timestamp in UTC. Render in the profile's timezone. Never store local times for events.
- Schedules, however, are defined in local wall-clock terms, because prescriptions are ("twice daily at 08:00 and 20:00"), and they must survive daylight saving transitions the way a human expects: the 08:00 dose stays at 08:00 on the wall clock.

- The double-dose guard, by contrast, must measure elapsed real time as UTC deltas, never wall-clock arithmetic. On the November fall-back night, wall-clock subtraction would tell you 9 hours passed when only 8 did. Schedule in local time; guard in absolute time. Write that sentence on an index card.
- New York observes DST. Test the spring-forward night (a 02:30 schedule that does not exist that day) and the fall-back night (a 01:30 that exists twice) explicitly. These four or five test cases are the project's crown jewels.

2. The double-dose guard. Pure function in the domain core: given a medication's minimum interval, its last confirmed dose time, and a proposed log time, return clear, warn, or duplicate. "Duplicate" covers the double-tap problem: two log attempts within a couple of minutes are almost certainly one human action, and the second should be absorbed rather than recorded as a second dose. Warnings that the user overrides are recorded as overridden (2.2), giving the practitioner report an honest picture rather than a sanitized one.

3. Missed-dose detection. A periodic evaluation (the scheduler invokes it; the core computes it) that asks: which DUE doses have exceeded grace without a log? Configuration per profile: grace minutes, alert repetition interval and count, response window before auto-MISSED, quiet hours, and a per-medication flag allowing a medication the user designates as critical to alert through quiet hours. The user makes that designation, not the software.

4. Data integrity. Append-only events with void-plus-reason instead of delete. Automatic timestamped backups of the SQLite file before any import or migration. A one-command full export (JSON) so her data is never hostage to your schema. WAL mode on SQLite for crash safety. These cost almost nothing now and are exactly the habits a security interviewer hopes to hear described.

4.5 Notification and alarm architecture per phase

Reminders are the feature most users judge a medication app by, and the honest truth is that each phase delivers them differently and imperfectly:

- **Phase 1 (CLI, your Mac).** Two viable mechanisms. Simplest: a check subcommand that evaluates due and missed doses, invoked every few minutes by macOS launchd (the native scheduler; cron also exists on macOS but launchd is the platform-correct tool), posting alerts via `osascript -e 'display notification ...'`. Alternative: a long-running daemon using APScheduler. Recommendation: launchd plus the check subcommand, because it teaches OS-level scheduling, survives reboots, and has no process to babysit. Limitation, stated plainly: Phase 1 alerts appear on your Mac, not on Lerato's phone. Phase 1's user is you; her interim reminders come from Apple Health's own Medications notifications, which she gets for free per 1.5.
- **Phase 3 (web and PWA).** Verified this session: iOS 16.4 and later supports Web Push for web apps that the user has added to their Home Screen from Safari; the permission prompt must be triggered by an explicit tap. This means a Django-served PWA can deliver real reminder notifications on her iPhone with no App Store presence. Requirements: HTTPS, a web app manifest, a service worker, VAPID keys, and a small push-subscription store. Fallback channel: email reminders via a transactional sender, configured per profile, because push subscriptions can silently die and a medication reminder system should never have a single delivery path.
- **Phase 4 (native iOS).** `UNUserNotificationCenter` local notifications, scheduled on-device, which is the gold standard for reliability: no server required for the reminder path at all. Time-sensitive notification treatment is worth investigating at build time; Apple's

Critical Alerts entitlement (which pierces silent mode) is restricted and requires special approval, so do not plan around it.

4.6 Apple Health integration: three lanes, verified

There is no public server-side API for Apple Health. Every integration path runs through her device. All three lanes below were verified this session except where flagged.

- **Lane A: the built-in export (Phase 1).** Health app, profile picture, Export All Health Data. Produces a zip containing export.xml and export_cda.xml. This is the official, free, manual lane. Two engineering realities: multi-year exports run to hundreds of megabytes or more, so parse with ElementTree's iterparse (streaming) rather than loading the tree; and the format is an undocumented database dump, so build the parser defensively and version it. Her medication logs and any wellbeing-adjacent samples ride along in the same export.
- **Lane B: Health Auto Export (Phase 2 to 3).** A maintained third-party iOS app that exports 150-plus Health metrics as JSON or CSV, including logged medication dosages and State of Mind data, with automations that can POST directly to a REST endpoint you control. REST automation sits in its premium tier (price not verified; check in-app). Constraints verified from its documentation: iOS forbids Health data access while the phone is locked, and automations depend on Background App Refresh, so deliveries are near-real-time at best, not guaranteed-instant. For this project's purposes (daily ingestion for stats and reports) that is entirely sufficient. An alternative app in the same niche, Health Export Pro, also exists; evaluate both at build time.
- **Lane C: native HealthKit (Phase 4).** Reading and writing Health data directly requires a native app with the HealthKit entitlement. This is also the only lane that can write back (App Store guideline 5.1.3(ii), verified: apps must not write false or inaccurate data into HealthKit). Third-party changelogs observed this session reference a recent medications data API in HealthKit; treat the exact API surface for reading medication records as a build-time verification item, flagged in Part 11.
- **Glue option, flagged not verified:** the iOS Shortcuts app has historically offered health-logging actions and the ability to call a URL, which could let a tap-one-button shortcut both log to Apple Health and POST to your Phase 3 API. Reasoned from established practice; verify the current action set on her iOS version before designing around it.

4.7 Privacy and security architecture

- **Local-first by default.** Phases 1 and 2 keep every byte on your machine. Phase 3 introduces a server; the privacy-maximal deployment is a small machine inside the household (or your own hardware) reachable over a private mesh network such as Tailscale rather than the open internet, with the public-internet deployment as the convenience alternative. Current terms and pricing of any mesh service are a build-time check.
- **Encryption posture, honestly.** Phase 1: rely on macOS FileVault full-disk encryption plus strict file permissions, and encrypt backups you move anywhere. Application-level SQLite encryption (SQLCipher and its Python bindings) is real but its packaging friction is nontrivial; treat it as a Phase 3 evaluation, flagged in Part 11, rather than a Phase 1 promise.

- **Notification content discipline.** Push and lock-screen notifications should support a discreet mode ("Scheduled reminder") rather than broadcasting medication names on a visible lock screen. Make it per-medication configurable.
- **No third parties.** No analytics SDKs, no ad frameworks, no data brokers, ever, in any phase. Verified guideline 5.1.3(i): health data may not be used for advertising or data mining; your architecture should make that violation impossible rather than merely avoided.
- **iCloud rule.** Verified guideline 5.1.3(ii): apps may not store personal health information in iCloud. Relevant only at Phase 4 and beyond, but it shapes the sync design early: household server sync, not iCloud sync.
- **Secrets hygiene.** VAPID keys, email-sender credentials, and any tokens live in environment configuration outside the repository from the first commit. The repository going public with a key in its history is an unforced error the security track teaches you to never make.

4.8 Technology selections, with reasoning

Decision	Selection	Reasoning
Language	Python 3.12+	The 2026 spine per the roadmap; everything here reinforces it
Storage	SQLite via sqlite3, WAL mode	Stdlib, zero ops, you hold SQL already; Django ORM supersedes the access layer in Phase 3, file format survives
CLI	argparse first	Stdlib, zero magic; Typer is a pleasant later upgrade, not a prerequisite
Charts	matplotlib for report PDFs; Plotly for the web	Exactly PCC chapters 15 to 17; static charts print better, interactive charts browse better
Report PDF	Jinja2 templates rendered by WeasyPrint	Reuses your held Jinja skill; HTML and CSS as layout language beats canvas-coordinate PDF building; ReportLab remains the fallback if WeasyPrint's system dependencies misbehave on a target machine
Web framework	Django (Phase 3)	PCC chapters 18 to 20 teach it; auth, ORM, forms, and the admin site (a free data-entry and inspection UI) ship in the box; Flask remains your known fallback; FastAPI enters only if Phase 4 wants a dedicated JSON API
Scheduler	launchd plus a check subcommand (Phase 1); in-process or system scheduler per host (Phase 3)	Platform-correct, reboot-surviving, teaches real ops
Native client	Swift and SwiftUI (Phase 4)	The mature path for a HealthKit, notification-heavy app. Python-on-iOS via BeeWare or Kivy is real and improving but rough for exactly this app's needs; reasoned from established practice, re-verify at the Phase 4 gate. The pragmatic split is a native Swift client speaking to your Python backend, which mirrors your six-app interface-plus-core philosophy with Python standing where C++ stood

Part 5: Phase Plan and Decision Gates

Hour figures are focused-work estimates at your current level, exclusive of the PCC reading they ride alongside. The governing constraint repeats: Q3's One Big Thing is the dual spine, Q4's is Security+ and the campaign. This project lives inside the Python study and Forge-adjacent hours and yields whenever they collide.

Phase 0: This week (2 hours maximum, no code)

1. Lerato begins logging her medications in Apple Health's built-in Medications feature today. Zero build cost, immediate data capture, free interim reminders.
2. Create the repository inside your existing Python workspace: pyproject.toml skeleton, README stub stating the problem, LICENSE decision deferred (Part 8.4), .gitignore, empty tests directory. No application code.
3. On paper, not in an editor: draw the data model from Part 4.2 in your own hand and walk the state machine from 4.3 aloud. Retrieval practice, applied.
4. Devlog entry number one ("Why I am building a medication tracker for my sister, a physician") becomes this week's public artifact.

Phase 1: CLI core (target window July to August 2026; 30 to 45 hours)

Rides alongside PCC chapters 9 to 11 plus the datetime and sqlite3 deep-study blocks.

Build order: domain entities; the time subsystem with its DST test suite; the dose state machine; the double-dose guard; SQLite repository; argparse commands (add medication, set schedule, log dose, log wellbeing, status, history); launchd check subcommand with osascript notifications; Apple Health export.xml importer (streaming); plain-text and first-pass PDF report.

Exit criteria (the gate): the DST and guard test suites pass and you can explain every case from memory; her real Apple Health export imports cleanly and idempotently twice in a row; a practitioner PDF for a real date range of her data prints legibly; the tool is installed on your machine via pipx and used daily for at least two weeks. Fail action: diagnose which subsystem stalled, cut every feature except medication logging plus the guard, and re-gate in three weeks.

Phase 2: Analytics and the practitioner report (September to October 2026; 20 to 30 hours)

Rides alongside PCC chapters 15 to 17 and serves as your Data Visualization project.

Build order: adherence statistics; wellbeing trend lines per axis; the adherence-versus-wellbeing overlay; report template v1 in Jinja2 plus WeasyPrint to the Part 6 specification; Health Auto Export ingestion endpoint stub or file-drop importer.

Exit criteria: Lerato hands a generated report to an actual practitioner, or reviews it herself wearing her physician hat, and you incorporate one full round of her corrections. The report is the product; her sign-off is the gate. Hard boundary: when the Security+ closure sprint begins, this project freezes wherever it stands until the exam is sat.

Phase 3: Web application and PWA reminders (December 2026 to Q1 2027; 40 to 60 hours)

Begins only after Security+ is sat, per the roadmap's Q4 gates. Rides alongside or after PCC chapters 18 to 20.

Build order: Django project with the existing domain core imported as a package; models mapped; auth (her account, your admin); daily-use mobile templates (log dose in two taps, log wellbeing in three); PWA manifest and service worker; Web Push with VAPID for due and missed alerts; email fallback channel; deployment to the household-server or small-host option chosen in 4.7.

Exit criteria: she logs from her own phone for two consecutive weeks without asking you for help; at least one real missed-dose push arrives on her iPhone; reports generate from the web UI. This is the phase where the project starts serving her daily rather than serving your learning primarily.

Phase 4: Native iOS companion (2027; 60 to 100+ hours; gated)

Enters only when the roadmap's 2027 posture allows a new technical front (Swift) and the Phase 3 system has proven insufficient in a specific, named way (typically: reminder reliability or HealthKit write-back). Requires the Apple Developer Program at 99 dollars per year, already a known pending item in your plan. Architecture: Swift and SwiftUI client; on-device UserNotifications for reminders; HealthKit read and write with her consent; your Python backend serving sync, stats, and reports. TestFlight with Lerato as the first external tester. Re-verify the Python-on-iOS landscape (BeeWare, Kivy) at this gate before committing to the Swift path, then commit fully to whichever wins.

Phase 5: Distribution decision (no date; gated by demand validation)

Public release is a separate product decision, not a default next step. The hot dog method applies: before App Store submission, validate that strangers want this (the devlog audience is the natural test bed: a waitlist post, a landing page, direct conversations). Pass: execute the Part 7 distribution checklist in full. Fail or unclear: the app remains a family tool and a portfolio piece, which is already complete success against this document's goals.

Android note

Health Connect is Android's HealthKit analogue and a native Kotlin commitment. Deferred indefinitely; revisit only if Phase 5 passes and Android demand is demonstrated. Recorded so it stops occupying mental space.

Gates summary

When	Gate	Pass	Fail
This week	Phase 0 complete inside 2 hours	Proceed with PCC as planned	Phase 0 was overthought; do steps 1 and 2 only
End August 2026	Phase 1 exit criteria	Begin Phase 2 with PCC ch. 15	Cut to guard-plus-logging core, re-gate 3 weeks
End October 2026	Practitioner-reviewed report shipped	Freeze for Security+ sprint with a clean tag	Freeze anyway; report becomes first Phase 3

			task
Security+ sat	Unfreeze decision	Phase 3 opens	Campaign and retake own the calendar; project stays frozen
Two weeks of her daily Phase 3 use	Web phase proven	Phase 4 evaluation opens for 2027	Fix the named friction before any native talk
Demand validated	Distribution	Part 7 checklist, then submit	Family tool plus portfolio piece; declared complete

Part 6: Practitioner Report Specification

Design this with Lerato as co-author; she is the rare first user who is also the report's professional audience.

Header block. Patient-entered display name; report date range; generation timestamp; report version; the data-provenance line: "All entries are patient self-reported via [app name]; medication logs may include data imported from Apple Health."

Section 1: Current regimen. Active medications with strength, form, schedule in plain words ("08:00 and 20:00 daily"), and regimen changes that occurred inside the range (ScheduleRule effective-dating makes this a query, not archaeology).

Section 2: Adherence summary. Per medication and overall: scheduled count, taken on time, taken late, missed, skipped with reasons, PRN usage count, and guard overrides if any. One small table, one bar or line chart. Percentages alongside raw counts, never instead of them; physicians distrust naked percentages.

Section 3: Dose event log. Chronological table for the range: date, time, medication, status, minutes late where applicable, note. Compact, complete, monospaced-friendly.

Section 4: Wellbeing trends. One chart per axis over the range with its scale and anchor labels printed; a compact table of weekly means and ranges. Multiple entries per day appear as daily mean with markers, never silently averaged away in the table.

Section 5: Overlay. Adherence events plotted against the wellbeing axes on a shared time axis. Present the picture; draw no conclusions in text. Correlation language stays out of the report entirely; the physician interprets, the report only shows.

Section 6: Notes. Free-text entries from the range that the patient marked as include-in-report. Default exclude; her notes are hers.

Footer on every page. The Part 2.2 disclaimer, page X of Y, generation timestamp.

Print discipline. US Letter; legible in grayscale (vary line styles and markers, never color alone); minimum 10 point body; charts sized for paper, not screens; no element split across a page break. Test by literally printing on the worst printer available.

Part 7: Compliance, Safety, and Distribution

While it is a personal and family tool (Phases 0 to 4): HIPAA does not apply; it binds covered entities such as providers, insurers, and their business associates, not individuals building software for their own household. You still behave as though the data deserves HIPAA-grade respect, because it does, and because describing that posture is portfolio gold.

The FDA position (verified this session): medication reminder software that prompts against user-predetermined schedules without treatment suggestions sits under FDA enforcement discretion. The bright lines that keep you there are in 2.2: user-defined everything, informational warnings, no interaction checking, no recommendations, persistent disclaimer. Crossing any of those lines is a project-redefining decision requiring real regulatory counsel, not a feature ticket.

App Store checklist (verified this session; applies at Phase 5):

- Guideline 5.1.3(i): health data never used for advertising, marketing, or data mining; never sold or shared with data brokers.
- Guideline 5.1.3(ii): no false or inaccurate writes to HealthKit; no personal health information stored in iCloud.
- Guideline 5.1.1(i): privacy policy linked in App Store Connect metadata and accessible inside the app.
- Account deletion available in-app if accounts exist.
- Notifications must not be required for core function and should avoid sensitive content (discreet mode from 4.7 satisfies this).
- Health-adjacent apps draw extra review scrutiny; unsubstantiated wellness claims are rejection bait. The app's copy describes what it records, never what it improves.

Disclaimer wording requirement (every phase, every surface): not a medical device; does not provide medical advice, diagnosis, or treatment; all data is self-reported; consult a qualified healthcare professional for medical decisions. Final wording is worth one hour with a template and, at Phase 5, a professional review.

If distribution ever goes international: GDPR and similar regimes treat health data as a special category. Flag, do not solve now; it is a Phase 5 work item with the rest of the checklist.

Part 8: Portfolio Packaging

8.1 Repository structure

A src-layout package (domain, services, persistence, interfaces as subpackages), tests mirroring the source tree, docs directory holding this document's successor and the report samples (with synthetic data, never her real data), pyproject.toml, README, CHANGELOG, LICENSE. The repository must be runnable by a stranger from the README alone; that is the bar.

8.2 README anatomy

The story first: built for my sister, a physician managing a medical condition, designed local-first because health data should not leave the household by default. Then: a screenshot or terminal capture, a sample report page rendered from synthetic data, the architecture diagram from 4.1, quickstart, the safety design section (state machine, guard, DST suite) written for a technical

reader, and a roadmap-of-phases section that shows deliberate staging. The safety design section is what differentiates this from every tutorial tracker on GitHub.

8.3 Engineering signals

pytest suite with the DST and guard cases prominent; coverage reported honestly; type hints with strict checking; Black and Ruff enforced (your June 9 environment already does this); GitHub Actions running the suite on push; conventional commit messages; semantic version tags from v0.1.0; an honest CHANGELOG. Each item is a five-minute habit that compounds into the difference between a repo and a portfolio piece.

8.4 License decision

Deferred but framed: MIT or Apache-2.0 maximizes portfolio reach and contribution; a proprietary license preserves Phase 5 commercial options but mutes the open-source signal. A common middle path is open core. Decide at the end of Phase 1 when the code's destiny is clearer; record the decision and reasoning in the devlog.

8.5 Naming

When replacing the codename: shortlist three to five candidates; check the App Store, Google Play, domain availability, and a USPTO trademark search before attachment forms; prefer names that survive being said aloud to a doctor. This is a one-evening task at the end of Phase 2, not before.

8.6 The interview narrative

For convergence roles, the honest framing: "I designed and shipped a safety-conscious health data system: append-only event ledger, timezone-correct scheduling with explicit DST testing, idempotent ingestion of a half-gigabyte undocumented XML export, local-first architecture with no third-party data flow, and a clinician-facing report as the primary deliverable." Every clause is a security-adjacent competency demonstrated, not claimed. The devlog series doubles as proof of consistent public output for the audience engine.

Part 9: Cost Ledger

All figures are planning estimates; verify prices when acted on.

Item	Phase	Cost	Notes
Phases 0 to 2 entirely	0 to 2	0 dollars	Stdlib, open-source libraries, your existing machine
Health Auto Export premium	2 to 3	Not verified	REST automation sits in its paid tier; check in-app; one-time lifetime option reportedly exists
Domain name	3	10 to 15 dollars per year, estimate	Optional if household-network deployment chosen
Small host or VPS	3	0 to 12 dollars per month, estimate	Free tiers exist and churn; household server option is 0
TLS certificate	3	0 dollars	Let's Encrypt

Apple Developer Program	4	99 dollars per year	Already a pending item in your master plan
Regulatory or legal review	5 only	Unknown	Only if public distribution proceeds

Nothing in Phases 0 to 2 competes with the 475 dollar certification budget. The first unavoidable spend is Phase 4's 99 dollars, which your plan already anticipates.

Part 10: Risk Ledger

Risk 1: Divergence, again, because it is always Risk 1. This project is designed to occupy the Python study hours, not add to them. The moment it starts borrowing from Google Cybersecurity Certificate hours or the Security+ sprint, it has become a front, and the One Big Thing rule applies: freeze it. The phase gates encode this, but gates only work if read.

Risk 2: Scope creep. Your six-app and six-game documentation history shows specs growing toward completeness. Here, completeness is the enemy of shipped. The v0.1 feature set is Part 2.1 items 1 through 4 plus a plain-text report. Everything else queues behind the Phase 1 gate.

Risk 3: Reminder reliability is the product. Every phase's delivery mechanism has a documented failure mode: launchd misconfiguration, dead push subscriptions, Background App Refresh throttling. Mitigations: the email fallback channel, a visible "last check ran at" heartbeat, and honest expectations with Lerato that until Phase 4, Apple's own Medications notifications remain her reliability backstop. A medication system must fail loudly, never silently.

Risk 4: The Phase 1 usefulness gap. The CLI serves your learning more than her daily life. The mitigation is structural (1.5): she logs in Apple Health from day one and your importer makes her data productive before your UI exists. Set that expectation with her explicitly this week so the project never feels, to her, like a promise deferred.

Risk 5: Medical liability discipline. One well-meaning feature (an interaction warning, a "consider taking with food" tip) silently crosses the 2.2 bright lines. Countermeasure: the out-of-scope list in 2.3 is reviewed at every phase gate, and any feature touching it requires this document to be formally revised first.

Risk 6: Apple platform dependency. Export formats, HealthKit surfaces, and PWA push behavior are Apple's to change. Mitigations: the importer is versioned and defensive; her data's canonical home is your SQLite ledger, not Apple's store; the full-export command guarantees portability forever.

Risk 7: Building past the need. It is possible Apple Health plus your report generator satisfies her completely by the end of Phase 2. If so, Phases 3 and 4 must justify themselves against real observed friction, not momentum. The gates ask for named friction for exactly this reason. Stopping at Phase 2 with a used tool and a reviewed report is a success outcome, not an abandonment.

Part 11: Verification Log

Per your anti-hallucination protocol: what was verified by web search in this session, and what is reasoned from established practice and flagged for build-time verification.

Verified this session:

- iOS and iPadOS 16.4 and later support Web Push for web apps added to the Home Screen via Safari; the permission request requires an explicit user gesture; push does not work from ordinary Safari tabs. (Multiple vendor documentation sources and Apple developer forum confirmation.)
- Apple Health's Export All Health Data produces a zip containing export.xml and export_cda.xml; multi-year exports commonly reach hundreds of megabytes. (Apple Support guide "Share your data in Health on iPhone"; corroborating tooling sites.)
- Apple Health includes a Medications feature with logging and a shareable medication list. (Apple Support.)
- Health Auto Export exists, is actively maintained, exports 150-plus metrics including logged medication dosages and State of Mind as JSON or CSV, supports REST API automations in its premium tier, and documents two constraints: no Health data access while the device is locked, and dependence on Background App Refresh. (App Store listing, developer documentation, third-party writeups.) Health Export Pro exists as an alternative.
- FDA policy: medication reminder software prompting against pre-determined, user-configured schedules without treatment suggestions falls under enforcement discretion. (FDA.gov, Policy for Device Software Functions and Mobile Medical Applications and its enforcement-discretion examples.)
- App Store Review Guidelines: 5.1.3(i) bars advertising and data-mining uses of health data; 5.1.3(ii) bars false HealthKit writes and bars storing personal health information in iCloud; 5.1.1(i) requires an accessible privacy policy. (Apple developer guidelines and corroborating sources.)

Reasoned from established practice; verify before relying on them:

- BeeWare and Kivy as Python-on-iOS options and their maturity limits for HealthKit-centric apps (re-verify at the Phase 4 gate).
- iOS Shortcuts health-logging and URL actions as a glue layer.
- The exact HealthKit API surface for reading medication records (third-party changelogs reference a recent medications data API; confirm the API name and availability when Phase 4 planning begins).
- Current feature sets of Medisafe and MyTherapy.
- SQLCipher Python binding state and packaging friction.
- Hosting free tiers, VPS pricing, Tailscale personal-plan terms, and Health Auto Export's current pricing.

Could not verify and did not assume: nothing in this document depends on an unverified load-bearing claim. Where a claim could not be verified, the dependent decision was deferred to a gate.

Part 12: Immediate Next Actions

1. **Today:** Lerato sets up her medications in Apple Health and turns on its reminders. Ten minutes, hers.
2. **This week, inside 2 hours:** Phase 0 steps 2 to 4. Repository skeleton, paper data model, devlog entry one.

3. **Ongoing:** PCC continues exactly as the roadmap prescribes. When chapter 9 arrives, this project's entities become your worked example.
4. **Standing working agreement, stated for the record:** when you hit walls in this build, the Stroustrup rule extends here at your request: design review, concept explanation, and Socratic hints are on the table; written solutions are not, unless you explicitly choose otherwise for a bounded piece. The tool is hers; the mastery is yours.

End of document. Version 1.0, June 12, 2026. Singular-document discipline applies: future updates fold into this document as versioned additions.